SYSTEM ARCHITECTURE DECK

Intelligent Candidate Discovery & Ranking

A Hybrid Multi-Stage Retrieval, NLI & CDF Pipeline

Presented by: team_404_Founders

Redrob AI Recruiter Platform Hackathon | July 2026

1. Core Philosophy & Design Trade-Offs

What We Built

- Hybrid Multi-Stage Ranker combining semantic embeddings, exact lexical matches, structured profile signals, and deep cross-attention reranking.
- Statistical CDF engine that normalizes raw candidate engagement numbers into unbiased relative percentiles, translating numerical signals into semantic text.
- Zero-Shot NLI Logical Gate to evaluate candidate location/mode preferences, preventing semantic embedding overlap from passing mismatched candidates.

Why We Built It This Way (Why / Why Not)

- Why Hybrid: Embeddings catch general capability; BM25 catches s
- Why Multi-Stage: Running a Cross-Encoder over 100K profiles on C Stage 1 Shortlist (top-2000) reduces the pool in milliseconds, enabli cross-encoder reranking (top-500) to execute in less than 20 secon
- Why Not Local LLMs: quantized 8B LLM inference takes ~40s per c This is completely infeasible under the 5-minute hackathon executio
- Why NLI over Hardcoding: Hardcoded filters are brittle and easily b formatting. NLI handles complex semantic contradiction checking o

2. System Architecture & Flow

OFFLINE PRECOMPUTATION

Runs once prior to ranking. Generates cache files.

1. Parse candidates.jsonl in multi-processed chunks
2. Compute CDF distributions for numerical signals
3. Build rich text representations incorporating relative percentiles and core career summaries
4. Generate 384-d dense embeddings using sentence-transformers/all-MiniLM-L6-v2
5. Compile BM25 Lexical Index (bm25_index.pkl)
6. Pre-screen profiles for temporal contradictions and gaming indicators (honeypot_flags.npy)
7. Compute base disqualifiers (disq_flags.npy)

ONLINE RANKING ENGINE (rank.py)

Executes on CPU inside Docker sandbox under 5 mins.

1. Load JD text from docx, clean, and embed
2. Retrieve candidate BM25 and structured scores
3. L1 Shortlist: Matrix multiplication of JD and 100K candidate embeddings (top-2000 shortlisted)
4. Stream top-2000 profiles & evaluate work mode/ relocation constraints via NLI logical model
5. Stage 1 Fusion (0.30 sem + 0.12 lex + 0.58 str)
6. Select top-500 and run token-level Cross-Encoder re-ranking using ms-marco-MiniLM-L-6-v2
7. Stage 2 Fusion (20% CE weight) & round scores
8. Stable lexicographical tie-break ID sorting

3. Innovation: NLI Logic Gate & CDF Statistics

Zero-Shot NLI Logical Constraint Gate

- Semantic similarity models group text by topic, confusing compatibility with relevance.
  For example, onsite requirements match highly with remote profiles due to keyword overlap.
- Solution: Deploy local cross-encoder/nli-deberta-v3-small on the L1 shortlist (top-2000).
- Method: Formulate logic gates as premise-hypothesis contradiction verification:
  - Premise: 'The candidate is willing to relocate to another city.'
  - Hypothesis: 'Candidate is not willing to relocate.'
- Outcome: High contradiction probability (P > 0.80) triggers a multiplicative penalty
  (factor of 0.10) to automatically drop incompatible candidates without hardcoding rules.

Statistical CDF-to-Text Normalization

- Standard systems apply arbitrary thresholds to numbers (e.g. response rate < 50% = bad).
- Solution: Build the Cumulative Distribution Function (CDF) across the entire pool of 100K.
- Process: Map candidate values to relative percentiles: Percentile(x) = (Count < x) / 100,000.
- Textualization: Convert percentiles to semantic categories: extremely low (0-10%), below average,
  average (25-75%), high, and exceptional. Assemble into a 'Behavioral Summary Query'
  (e.g., 'Candidate shows high engagement with recruiters. Candidate average response time is exceptional.')
  and append to candidate resume text before embedding. AI naturally reads and weights these signals.

4. Anti-Gaming, Robustness & Trap Protection

Skill Trust Score (Anti-Keyword Stuffing)

- The Trap: Candidates stuff their profile with expert keywords they've never used.
- The Defense: Implement a multi-dimensional trust multiplier for each declared skill:

  Trust = Proficiency Map x Duration Months x Endorsement Count x Assessment Score
- A candidate with verified assessments and endorsements ranks significantly higher

  than a keyword stuffer who lists 20 expert skills with 0 months of experience.
- Text building also weights career history descriptions above raw skills lists.

Honeypot Screening & Multiplicative Disqualifiers

- Honeypot Screening: 7-point validation check screens out seeded impossible profiles

  (e.g. concurrent overlapping full-time roles, experience length exceeding age, expert claims

  with near-zero assessment). 3+ flags triggers a 0.0 multiplier (hard mask), dropping them instantly.
- Multiplicative Disqualifiers: Hard constraints are applied as scaling factors (not additive):

  - Consulting-Firm-Only Career: candidates with only TCS/Infosys etc. experience get a 0.10 multiplier.

  - Title-Description Mismatch: marketing/sales titles stuffed with AI keywords get a 0.20 multiplier.

  - Behavioral Zombies: inactive candidates (last active > 150 days) receive a 0.35 multiplier.

  - Notice Period: notice periods exceeding 90 days receive a 0.80 multiplier.

5. Performance Optimizations & Compliance

Compute Constraints & Code Reproduction

- CPU-Only, No Network, 16GB RAM: Fully complied. No external API calls are made.
- Model Choice: Switched bi-encoder from nomic-embed-text-v1.5 (12 hours to embed 100K on CPU) to sentence-transformers/all-MiniLM-L6-v2 (50 minutes to embed 100K on CPU).
- Execution: Matrix dot product (100K candidates x 384 dimensions) completes in <1s.
- Memory Efficiency: Dense embeddings loaded using NumPy memory-mapping (mmap_mode='r'), ensuring a minimal memory footprint (< 4GB RAM) and sub-second CLI start-up time.

Submission Verification & Results

- Pipeline Execution Speed: rank.py completes in 120.1 seconds on standard CPU.
- Output Format: Validated using validate_submission.py, ensuring 100 unique candidates, monotonically non-increasing scores, and ascending lexicographical tie-breakers.
- Deployments: Pushed code to GitHub (Adityakeerti/resumeRankerAi) and live Docker-based demo to Hugging Face Spaces (adityacodes404/redrob-ranker) with Streamlit UI.
- Codebase structure: Organized cleanly into modular files for structured scoring, honeypot detection, BM25 indexing, semantic bi-encoder/cross-encoder, and reasoning.